```
import { useState, useEffect } from "react";

const YEARS = Array.from({ length: 35 }, (_, i) => 2024 - i);
const MAKES = ["Acura","BMW","Buick","Cadillac","Chevrolet","Chrysler","Dodge","F
const MODELS = {
  Ford: ["F-150","Mustang","Explorer","Escape","Edge","Fusion","Focus","Bronco","
  Chevrolet: ["Silverado","Equinox","Malibu","Traverse","Tahoe","Camaro","Colorad
  Toyota: ["Camry","Corolla","RAV4","Tacoma","Tundra","Highlander","4Runner","Pri
  Honda: ["Civic","Accord","CR-V","Pilot","Odyssey","HR-V","Ridgeline","Passport"
  Nissan: ["Altima","Sentra","Rogue","Pathfinder","Frontier","Murano","Maxima","A
  Dodge: ["Ram 1500","Charger","Challenger","Durango","Journey","Grand Caravan","
  Jeep: ["Wrangler","Grand Cherokee","Cherokee","Compass","Gladiator","Renegade",
  BMW: ["3 Series","5 Series","7 Series","X3","X5","X7","M3","M5","4 Series","2 S
  "Mercedes-Benz": ["C-Class","E-Class","S-Class","GLE","GLC","GLS","A-Class","CL
  Hyundai: ["Elantra","Sonata","Tucson","Santa Fe","Palisade","Kona","Ioniq","Vel
  Kia: ["Sorento","Sportage","Telluride","Soul","Forte","Stinger","Carnival","Sel
  Subaru: ["Outback","Forester","Crosstrek","Impreza","Legacy","Ascent","WRX","BR
  GMC: ["Sierra","Terrain","Acadia","Yukon","Canyon","Envoy","Jimmy"],
  Ram: ["1500","2500","3500","ProMaster","Dakota"],
  Mazda: ["Mazda3","Mazda6","CX-5","CX-9","CX-30","MX-5 Miata","CX-50"],
  Volkswagen: ["Jetta","Passat","Tiguan","Atlas","Golf","GTI","ID.4","Taos"],
  Lexus: ["RX","ES","NX","GX","IS","LS","UX","LC"],
  Acura: ["MDX","RDX","TLX","ILX","NSX","RLX"],
  Buick: ["Enclave","Encore","LaCrosse","Verano","Envision"],
  Cadillac: ["Escalade","XT5","CT5","CT4","XT4","XT6"],
  Chrysler: ["300","Pacifica","Voyager"],
  Lincoln: ["Navigator","Aviator","Corsair","Nautilus","Continental"],
  Infiniti: ["Q50","Q60","QX60","QX80","QX50"],
  Mitsubishi: ["Outlander","Eclipse Cross","Galant","Lancer","Mirage"],
  Volvo: ["XC90","XC60","S60","V60","XC40","S90"],
  Tesla: ["Model 3","Model Y","Model S","Model X","Cybertruck"],
};

const DTC_DATABASE = {
  "P0300": { code:"P0300", full_name:"Random/Multiple Cylinder Misfire Detected",
  "P0420": { code:"P0420", full_name:"Catalyst System Efficiency Below Threshold
  "P0171": { code:"P0171", full_name:"System Too Lean (Bank 1)", system:"Engine /
  "P0442": { code:"P0442", full_name:"Evaporative Emission System Small Leak Dete
  "P0128": { code:"P0128", full_name:"Coolant Thermostat (Coolant Temperature Bel
  "P0562": { code:"P0562", full_name:"System Voltage Low", system:"Electrical / C
  "P0700": { code:"P0700", full_name:"Transmission Control System Malfunction", s
};

const TORQUE_DATABASE = {
```

```
  "lug nuts": { component:"Lug Nuts / Wheel Bolts", vehicle_note:"ALWAYS verify w
  "spark plugs": { component:"Spark Plugs", vehicle_note:"Torque varies by plug t
  "oil drain plug": { component:"Oil Drain Plug", vehicle_note:"Generic ranges. C
};

const FLUID_DATABASE = {
  "honda civic": { vehicle:"Honda Civic", engine_note:"Capacities vary by engine.
  "toyota camry": { vehicle:"Toyota Camry", engine_note:"Common engines: 2.5L 4-c
  "ford f-150": { vehicle:"Ford F-150", engine_note:"Common engines: 2.7L EcoBoos
};

const COMPLAINT_DATABASE = [
  { match:{make:"Toyota",model:"Corolla",year_min:2009,year_max:2013,keywords:["a
  { match:{keywords:["misfire","rough idle","engine shake","rough running","stumb
  { match:{keywords:["won't start","wont start","no start","cranks but won't star
  { match:{keywords:["battery drain","battery dies","battery dead overnight","par
  { match:{keywords:["brake squeal","brake squealing","squealing brakes","brake n
  { match:{make:"Subaru",keywords:["head gasket","coolant loss","losing coolant",
  { match:{keywords:["transmission slipping","trans slipping","slipping","rpm rev
  { match:{keywords:["overheating","overheats","running hot","temp gauge high"]},
];

function complaintLookup(year, make, model, complaint) {
  if (!year || !make || !model || !complaint) return null;
  const yearNum = parseInt(year);
  const complaintLower = complaint.toLowerCase();
  let bestMatch = null, bestScore = -1;
  for (const entry of COMPLAINT_DATABASE) {
    const m = entry.match;
    let score = 0;
    if (m.make && m.make.toLowerCase() !== make.toLowerCase()) continue;
    if (m.make) score += 50;
    if (m.model && m.model.toLowerCase() !== model.toLowerCase()) continue;
    if (m.models && !m.models.some(mm => mm.toLowerCase() === model.toLowerCase()
    if (m.model || m.models) score += 50;
    if (m.year_min && yearNum < m.year_min) continue;
    if (m.year_max && yearNum > m.year_max) continue;
    if (m.year_min || m.year_max) score += 30;
    if (m.keywords) {
      const matched = m.keywords.filter(kw => complaintLower.includes(kw.toLowerC
      if (matched.length === 0) continue;
      score += matched.length * 10;
    }
    if (score > bestScore) { bestScore = score; bestMatch = entry; }
  }
  return bestMatch;
}
```

```
function fluidLookup(year, make, model) {
  if (!make || !model) return null;
  const key = (make + " " + model).toLowerCase().trim();
  for (const dbKey of Object.keys(FLUID_DATABASE)) {
    if (key.includes(dbKey) || dbKey === key) return FLUID_DATABASE[dbKey];
  }
  return null;
}

function dtcLookup(code) {
  return DTC_DATABASE[code.toUpperCase().trim()] || null;
}

function torqueLookup(component) {
  const key = component.toLowerCase().trim();
  for (const dbKey of Object.keys(TORQUE_DATABASE)) {
    if (key.includes(dbKey) || dbKey.includes(key)) return TORQUE_DATABASE[dbKey]
  }
  return null;
}

function extractJSON(text) {
  if (!text) throw new Error("Empty response");
  try { return JSON.parse(text); } catch {}
  let cleaned = text.replace(/```json\s*/gi, "").replace(/```\s*/g, "").trim();
  try { return JSON.parse(cleaned); } catch {}
  const start = cleaned.indexOf("{"), end = cleaned.lastIndexOf("}");
  if (start !== -1 && end > start) {
    try { return JSON.parse(cleaned.slice(start, end + 1)); } catch {}
    try {
      const repaired = cleaned.slice(start, end + 1)
        .replace(/[\u201C\u201D]/g, '"').replace(/[\u2018\u2019]/g, "'")
        .replace(/,(\s*[}\]])/g, "$1").replace(/\r?\n/g, "\\n");
      return JSON.parse(repaired);
    } catch {}
  }
  throw new Error("Could not parse AI response");
}

async function callClaude(prompt, systemPrompt) {
  const response = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: 1500,
```

```
      system: systemPrompt + "\n\nCRITICAL: Return ONLY raw JSON. No markdown, no
      messages: [{ role: "user", content: prompt }],
    }),
  });
  if (!response.ok) throw new Error(`API ${response.status}`);
  const data = await response.json();
  if (data.error) throw new Error(data.error.message);
  return data.content?.[0]?.text || "";
}

// ── Shared Styles ──
const S = {
  label: { display:"block", fontFamily:"'Courier New', monospace", fontSize:10, l
  body: { color:"#aaa", fontSize:13, lineHeight:1.7, fontFamily:"'Georgia', serif
  badge: { display:"inline-block", padding:"3px 8px", borderRadius:2, fontSize:10
  btn: { background:"#e8a020", color:"#000", border:"none", padding:"12px 28px",
  select: { background:"#111", border:"1px solid #333", color:"#e8e8e8", padding:
};

function Spinner({ label = "Processing..." }) {
  return (
    <div style={{ display:"flex", alignItems:"center", gap:10, color:"#e8a020", f
      <div className="spinner" /><span>{label}</span>
    </div>
  );
}

function Card({ title, children, accent = "#333" }) {
  return (
    <div style={{ background:"#0d0d0d", border:`1px solid ${accent}`, borderRadiu
      <div style={{ position:"absolute", top:0, left:0, right:0, height:2, backgr
      {title && <div style={{ color:accent, fontFamily:"'Courier New', monospace"
      {children}
    </div>
  );
}

function ErrMsg({ msg }) {
  return (
    <div style={{ marginTop:16, padding:16, background:"#1a0a0a", border:"1px sol
      <div style={{ color:"#e84040", fontFamily:"monospace", fontSize:12, marginB
      <div style={{ color:"#ff8080", fontFamily:"monospace", fontSize:12 }}>{msg
      <div style={{ color:"#888", fontFamily:"monospace", fontSize:11, marginTop:
    </div>
  );
}
```

```
function DiffBadge({ d }) {
  const cfg = { Easy:["#1a3a1a","#50d050"], Medium:["#3a2a0a","#e8a020"], Hard:["
  const [bg, fg] = cfg[d] || ["#1a1a1a","#888"];
  return <span style={{ ...S.badge, background:bg, color:fg }}>{d || "—"}</span>;
}

function VehicleSelector({ year, setYear, make, setMake, model, setModel }) {
  const models = make ? (MODELS[make] || []) : [];
  return (
    <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:12, marg
      {[
        { label:"YEAR", val:year, set:setYear, opts:YEARS.map(y => ({val:y, label
        { label:"MAKE", val:make, set:(v) => { setMake(v); setModel(""); }, opts:
        { label:"MODEL", val:model, set:setModel, opts:models.map(m => ({val:m, l
      ].map(({ label, val, set, opts, disabled }) => (
        <div key={label}>
          <label style={S.label}>{label}</label>
          <select style={{ ...S.select, opacity: disabled ? 0.4 : 1 }} value={val
            <option value="">Any</option>
            {opts.map(o => <option key={o.val} value={o.val}>{o.label}</option>)}
          </select>
        </div>
      ))}
    </div>
  );
}

// ── Tab 1: Complaint ──
function ComplaintLookup() {
  const [year, setYear] = useState(""); const [make, setMake] = useState(""); con
  const [complaint, setComplaint] = useState(""); const [loading, setLoading] = u

  async function handle() {
    if (!year || !make || !model || !complaint.trim()) return;
    setLoading(true); setResult(null);
    const db = complaintLookup(year, make, model, complaint);
    if (db) { setResult(db); setLoading(false); return; }
    try {
      const raw = await callClaude(`Vehicle: ${year} ${make} ${model}\nComplaint:
        `You are an expert automotive technician. Respond ONLY with a JSON object
      setResult({ ...extractJSON(raw), _source:"ai" });
    } catch (e) { setResult({ error:true, msg:e.message }); }
    setLoading(false);
  }

  return (
    <div>
```

```
      <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
      <div style={{ marginBottom:14 }}>
        <label style={S.label}>COMPLAINT / SYMPTOM</label>
        <textarea style={{ ...S.select, minHeight:80, resize:"vertical", lineHeig
          placeholder="Describe the issue — e.g., 'A/C blows warm, compressor clu
          value={complaint} onChange={e => setComplaint(e.target.value)} />
      </div>
      <button onClick={handle} disabled={loading || !year || !make || !model || !
        {loading ? "DIAGNOSING..." : "▶ RUN DIAGNOSIS"}
      </button>
      {loading && <Spinner label="Diagnosing..." />}
      {result?.error && <ErrMsg msg={result.msg} />}
      {result && !result.error && <>
        <Card title="COMPLAINT SUMMARY" accent="#e8a020">
          {result._source === "database" && <span style={{ ...S.badge, background
          {result._source === "ai" && <span style={{ ...S.badge, background:"#1a1
          <p style={S.body}>{result.complaint_summary}</p>
        </Card>
        {result.safety_warning && <Card title="⚠ SAFETY WARNING" accent="#e83030"
        <Card title="LIKELY CAUSES" accent="#20a0e8">
          <ol style={{ paddingLeft:18, margin:0 }}>{result.likely_causes?.map((c,
        </Card>
        <Card title="DIAGNOSIS STEPS" accent="#20e8a0">
          <ol style={{ paddingLeft:18, margin:0 }}>{result.diagnosis_steps?.map((
        </Card>
        <Card title="CORRECTIONS" accent="#e8a020">
          {result.corrections?.map((c,i) => (
            <div key={i} style={{ marginBottom:14, paddingBottom:14, borderBottom
              <div style={{ display:"flex", justifyContent:"space-between", margi
                <span style={{ color:"#fff", fontWeight:600, fontFamily:"monospac
                <div style={{ display:"flex", gap:8 }}>
                  <DiffBadge d={c.difficulty} />
                  {c.est_cost && <span style={{ ...S.badge, background:"#1a1a2a",
                </div>
              </div>
              <p style={{ ...S.body, margin:0 }}>{c.detail}</p>
            </div>
          ))}
        </Card>
        {result.tech_notes && <Card title="TECH NOTES / TSBs" accent="#a020e8"><p
      </>}
    </div>
  );
}

// ── Tab 2: DTC ──
function DTCLookup() {
```

```
const [code, setCode] = useState(""); const [year, setYear] = useState(""); con
const [loading, setLoading] = useState(false); const [result, setResult] = useS

async function handle() {
  if (!code.trim()) return;
  setLoading(true); setResult(null);
  const db = dtcLookup(code);
  if (db) { setResult({ ...db, _source:"database" }); setLoading(false); return
  const v = year && make && model ? `${year} ${make} ${model}` : "generic vehic
  try {
    const raw = await callClaude(`DTC Code: ${code.toUpperCase()}\nVehicle: ${v
      `You are an expert OBD-II specialist. Respond ONLY with JSON: {"code":"..
    setResult({ ...extractJSON(raw), _source:"ai" });
  } catch (e) { setResult({ error:true, msg:e.message }); }
  setLoading(false);
}

return (
  <div>
    <div style={{ marginBottom:14 }}>
      <label style={S.label}>DTC CODE</label>
      <input style={{ ...S.select, textTransform:"uppercase", letterSpacing:2,
        placeholder="e.g. P0300, B1234, C0035, U0100"
        value={code} onChange={e => setCode(e.target.value)} onKeyDown={e => e.
    </div>
    <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
    <button onClick={handle} disabled={loading || !code.trim()} style={S.btn}>{
    {loading && <Spinner label="Decoding DTC..." />}
    {result?.error && <ErrMsg msg={result.msg} />}
    {result && !result.error && <>
      <Card accent="#e8a020">
        <div style={{ display:"flex", justifyContent:"space-between", alignItem
          <div>
            <div style={{ display:"flex", alignItems:"center", gap:10, flexWrap
              <div style={{ fontFamily:"monospace", fontSize:28, fontWeight:700
              {result._source === "database" && <span style={{ ...S.badge, back
              {result._source === "ai" && <span style={{ ...S.badge, background
            </div>
            <div style={{ color:"#fff", fontSize:14 }}>{result.full_name}</div>
            <div style={{ color:"#666", fontSize:11, fontFamily:"monospace", ma
          </div>
          <div style={{ padding:"8px 16px", borderRadius:3, fontFamily:"monospa
            {result.drive_safe ? "✓ DRIVEABLE" : "✗ DO NOT DRIVE"}
          </div>
        </div>
        <p style={{ ...S.body, marginTop:12 }}>{result.drive_note}</p>
      </Card>
```

```
        <Card title="DESCRIPTION" accent="#20a0e8"><p style={S.body}>{result.desc
        <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12 }}>
          <Card title="COMMON CAUSES" accent="#e87020">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.common_causes?.map((
          </Card>
          <Card title="SYMPTOMS" accent="#a020e8">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.symptoms?.map((s,i)
          </Card>
        </div>
        <Card title="DIAGNOSIS PROCEDURE" accent="#20e8a0">
          <ol style={{ paddingLeft:18, margin:0 }}>{result.diagnosis_procedure?.m
        </Card>
        <Card title="COMMON FIXES" accent="#e8a020">
          {result.common_fixes?.map((f,i) => (
            <div key={i} style={{ marginBottom:14, paddingBottom:14, borderBottom
              <div style={{ display:"flex", justifyContent:"space-between", margi
                <span style={{ color:"#fff", fontFamily:"monospace", fontSize:14
                <DiffBadge d={f.difficulty} />
              </div>
              <p style={{ ...S.body, margin:0 }}>{f.notes}</p>
            </div>
          ))}
        </Card>
        {result.related_codes?.length > 0 && (
          <Card title="RELATED CODES" accent="#555">
            <div style={{ display:"flex", gap:8, flexWrap:"wrap" }}>
              {result.related_codes.map(c => (
                <button key={c} onClick={() => { setCode(c); setResult(null); }}
                  style={{ ...S.badge, cursor:"pointer", background:"#1a1a1a", co
                  {c}
                </button>
              ))}
            </div>
          </Card>
        )}
      </>}
    </div>
  );
}

// ── Tab 3: Torque Specs ──
function TorqueSpecs() {
  const [year, setYear] = useState(""); const [make, setMake] = useState(""); con
  const [engine, setEngine] = useState(""); const [component, setComponent] = use
  const [loading, setLoading] = useState(false); const [result, setResult] = useS
  const quickPicks = ["Lug Nuts / Wheel Bolts","Spark Plugs","Oil Drain Plug","Cy
```

```
async function handle(comp) {
  const c = comp || component;
  if (!c.trim()) return;
  if (comp) setComponent(comp);
  setLoading(true); setResult(null);
  const db = torqueLookup(c);
  if (db) { setResult({ ...db, _source:"database" }); setLoading(false); return
  const v = year && make && model ? `${year} ${make} ${model}${engine?" "+engin
  try {
    const raw = await callClaude(`Vehicle: ${v}\nComponent: ${c}`,
      `You are an expert automotive technician. Respond ONLY with JSON: {"compo
    setResult({ ...extractJSON(raw), _source:"ai" });
  } catch (e) { setResult({ error:true, msg:e.message }); }
  setLoading(false);
}

return (
  <div>
    <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
    <div style={{ marginBottom:12 }}>
      <label style={S.label}>ENGINE (optional)</label>
      <input style={S.select} placeholder="e.g. 5.7L HEMI, 2.0T, 3.5L EcoBoost"
    </div>
    <div style={{ marginBottom:12 }}>
      <label style={S.label}>COMPONENT / FASTENER</label>
      <input style={S.select} placeholder="e.g. lug nuts, cylinder head bolts,
        value={component} onChange={e => setComponent(e.target.value)} onKeyDow
    </div>
    <div style={{ marginBottom:16 }}>
      <div style={{ fontFamily:"monospace", fontSize:10, letterSpacing:3, color
      <div style={{ display:"flex", flexWrap:"wrap", gap:6 }}>
        {quickPicks.map(q => (
          <button key={q} onClick={() => handle(q)}
            style={{ ...S.badge, background:"#111", color:"#777", border:"1px s
            {q}
          </button>
        ))}
      </div>
    </div>
    <button onClick={() => handle()} disabled={loading || !component.trim()} st
    {loading && <Spinner label="Fetching torque specifications..." />}
    {result?.error && <ErrMsg msg={result.msg} />}
    {result && !result.error && <>
      <Card accent="#e8a020">
        <div style={{ display:"flex", alignItems:"center", gap:10, flexWrap:"wr
          <div style={{ color:"#fff", fontSize:18, fontWeight:700 }}>{result.co
          {result._source === "database" && <span style={{ ...S.badge, backgrou
```

```
    {result._source === "ai" && <span style={{ ...S.badge, background:"#1
  </div>
  {result.vehicle_note && <p style={S.body}>{result.vehicle_note}</p>}
</Card>
{result.critical_warnings?.length > 0 && (
  <Card title="⚠ CRITICAL WARNINGS" accent="#e83030">
    {result.critical_warnings.map((w,i) => (
      <div key={i} style={{ display:"flex", gap:10, marginBottom:6 }}>
        <span style={{ color:"#e84040", flexShrink:0 }}>!</span>
        <p style={{ ...S.body, color:"#ff8080", margin:0 }}>{w}</p>
      </div>
    ))}
  </Card>
)}
<Card title="TORQUE SPECIFICATIONS" accent="#20a0e8">
  <div style={{ overflowX:"auto" }}>
    <table style={{ width:"100%", borderCollapse:"collapse", fontFamily:"
      <thead>
        <tr style={{ borderBottom:"1px solid #333" }}>
          {["FASTENER","LOCATION","FT-LB","Nm","NOTES"].map(h => (
            <th key={h} style={{ textAlign:"left", padding:"6px 10px", co
          ))}
        </tr>
      </thead>
      <tbody>
        {result.specs?.map((s,i) => (
          <tr key={i} style={{ borderBottom:"1px solid #1a1a1a" }}>
            <td style={{ padding:"10px", color:"#e8e8e8" }}>{s.fastener}<
            <td style={{ padding:"10px", color:"#777", fontSize:11 }}>{s.
            <td style={{ padding:"10px", color:"#e8a020", fontWeight:700,
            <td style={{ padding:"10px", color:"#20a0e8" }}>{s.torque_nm}
            <td style={{ padding:"10px", color:"#777", fontSize:11 }}>{s.
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</Card>
{result.torque_sequence_description && <Card title="TORQUE SEQUENCE" acce
<div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12 }}>
  <Card title="THREAD INFO & TOOL" accent="#a020e8">
    <p style={S.body}>{result.thread_info}</p>
    <div style={{ marginTop:10 }}>
      <div style={{ ...S.label, marginBottom:4 }}>RECOMMENDED WRENCH</div
      <div style={{ color:"#a060e8", fontFamily:"monospace", fontSize:12
    </div>
  </Card>
```

```
          <Card title="COMMON MISTAKES" accent="#e84040">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.common_mistakes?.map
          </Card>
        </div>
      </>}
    </div>
  );
}

// ── Tab 4: Fluid Capacity ──
function FluidCapacity() {
  const [year, setYear] = useState(""); const [make, setMake] = useState(""); con
  const [engine, setEngine] = useState(""); const [trans, setTrans] = useState(""
  const [loading, setLoading] = useState(false); const [result, setResult] = useS

  const fluidColors = { "engine oil":"#e8a020", "coolant":"#20e8a0", "transmissio
  function getColor(sys) {
    const low = sys.toLowerCase();
    for (const [k,v] of Object.entries(fluidColors)) if (low.includes(k)) return
    return "#888";
  }

  async function handle() {
    setLoading(true); setResult(null);
    const db = fluidLookup(year, make, model);
    if (db) { setResult({ ...db, _source:"database" }); setLoading(false); return
    const v = year && make && model ? `${year} ${make} ${model}${engine?" "+engin
    try {
      const raw = await callClaude(`Vehicle: ${v}`,
        `You are an automotive fluid expert. Respond ONLY with JSON: {"vehicle":"
      setResult({ ...extractJSON(raw), _source:"ai" });
    } catch (e) { setResult({ error:true, msg:e.message }); }
    setLoading(false);
  }

  return (
    <div>
      <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
      <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12, margin
        <div>
          <label style={S.label}>ENGINE (optional)</label>
          <input style={S.select} placeholder="e.g. 3.5L V6, 2.0T, 5.7L HEMI" val
        </div>
        <div>
          <label style={S.label}>TRANSMISSION (optional)</label>
          <select style={S.select} value={trans} onChange={e => setTrans(e.target
            {["","Automatic","Manual","CVT","DCT / Dual-Clutch","4WD / AWD"].map(
```

```
    </select>
  </div>
</div>
<button onClick={handle} disabled={loading} style={S.btn}>{loading ? "LOADI
{loading && <Spinner label="Retrieving fluid specifications..." />}
{result?.error && <ErrMsg msg={result.msg} />}
{result && !result.error && <>
  <Card accent="#20e8a0">
    <div style={{ display:"flex", alignItems:"center", gap:10, flexWrap:"wr
      <div style={{ color:"#fff", fontSize:16, fontWeight:700 }}>{result.ve
      {result._source === "database" && <span style={{ ...S.badge, backgrou
      {result._source === "ai" && <span style={{ ...S.badge, background:"#1
    </div>
    {result.engine_note && <p style={S.body}>{result.engine_note}</p>}
  </Card>
  <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12, marg
    {result.fluids?.map((f,i) => {
      const color = getColor(f.system);
      return (
        <div key={i} style={{ background:"#0d0d0d", border:`1px solid ${col
          <div style={{ position:"absolute", top:0, left:0, right:0, height
          <div style={{ display:"flex", justifyContent:"space-between", ali
            <div style={{ fontFamily:"monospace", fontSize:10, letterSpacin
            <div style={{ fontFamily:"monospace", fontSize:16, fontWeight:7
          </div>
          <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:
            {[["Spec",f.spec],["Grade",f.viscosity_grade||"—"],["Type",f.fl
              <div key={lbl}>
                <div style={{ fontFamily:"monospace", fontSize:9, color:"#4
                <div style={{ fontFamily:"monospace", fontSize:11, color:"#
              </div>
            ))}
          </div>
          {f.notes && <p style={{ ...S.body, fontSize:11, margin:0, color:"
        </div>
      );
    })}
  </div>
  <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12, marg
    {result.drain_plug_torque && (
      <Card title="DRAIN PLUG TORQUE" accent="#e8a020">
        <div style={{ fontFamily:"monospace", fontSize:20, fontWeight:700,
      </Card>
    )}
    {result.filter_notes && <Card title="FILTER NOTES" accent="#20a0e8"><p
  </div>
  {result.special_considerations && (
```

```
          <Card title="SPECIAL CONSIDERATIONS" accent="#e84040">
            <p style={{ ...S.body, color:"#ff9090" }}>{result.special_considerati
          </Card>
        )}
        {result.maintenance_tips?.length > 0 && (
          <Card title="MAINTENANCE TIPS" accent="#e8e020">
            {result.maintenance_tips.map((t,i) => (
              <div key={i} style={{ display:"flex", gap:10, marginBottom:8 }}>
                <span style={{ color:"#e8e020", flexShrink:0 }}>✦</span><span sty
              </div>
            ))}
          </Card>
        )}
      </>}
    </div>
  );
}

// ── Tab 5: Parts Estimator ──
function PartsEstimator() {
  const [year, setYear] = useState(""); const [make, setMake] = useState(""); con
  const [engine, setEngine] = useState(""); const [job, setJob] = useState(""); c
  const [loading, setLoading] = useState(false); const [result, setResult] = useS

  async function handle() {
    if (!job.trim()) return;
    setLoading(true); setResult(null);
    const v = year && make && model ? `${year} ${make} ${model}${engine?" "+engin
    try {
      const raw = await callClaude(`Vehicle: ${v}\nJob: ${job}\nPart Quality: ${q
        `You are an automotive parts specialist. Respond ONLY with JSON: {"job_ti
      setResult(extractJSON(raw));
    } catch (e) { setResult({ error:true, msg:e.message }); }
    setLoading(false);
  }

  return (
    <div>
      <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
      <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12, margin
        <div>
          <label style={S.label}>ENGINE (optional)</label>
          <input style={S.select} placeholder="e.g. 5.0L V8, 2.5L 4-cyl" value={e
        </div>
        <div>
          <label style={S.label}>PART QUALITY</label>
          <select style={S.select} value={quality} onChange={e => setQuality(e.ta
```

```
      {["OEM","OEM Equivalent","Aftermarket","Remanufactured","Budget"].map
    </select>
  </div>
</div>
<div style={{ marginBottom:14 }}>
  <label style={S.label}>REPAIR JOB</label>
  <input style={S.select} placeholder="e.g. front brake pads and rotors, ti
    value={job} onChange={e => setJob(e.target.value)} onKeyDown={e => e.ke
</div>
<button onClick={handle} disabled={loading || !job.trim()} style={S.btn}>{l
{loading && <Spinner label="Estimating parts & costs..." />}
{result?.error && <ErrMsg msg={result.msg} />}
{result && !result.error && <>
  <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:12,
    {[
      { label:"DIY TOTAL", value:result.total_diy_cost, color:"#20e870", bg
      { label:"SHOP TOTAL", value:result.total_shop_cost, color:"#e84040",
      { label:"YOU SAVE", value:result.savings, color:"#e8e020", bg:"#2a2a0
    ].map(item => (
      <div key={item.label} style={{ background:item.bg, border:`1px solid
        <div style={{ ...S.label, marginBottom:6 }}>{item.label}</div>
        <div style={{ fontFamily:"monospace", fontSize:22, fontWeight:700,
      </div>
    ))}
  </div>
  <Card title="PARTS LIST" accent="#20a0e8">
    <div style={{ overflowX:"auto" }}>
      <table style={{ width:"100%", borderCollapse:"collapse", fontFamily:"
        <thead>
          <tr style={{ borderBottom:"1px solid #333" }}>
            {["PART","P/N","QTY","OEM","AFTERMKT","CRITICAL"].map(h => (
              <th key={h} style={{ textAlign:"left", padding:"6px 10px", co
            ))}
          </tr>
        </thead>
        <tbody>
          {result.parts?.map((p,i) => (
            <tr key={i} style={{ borderBottom:"1px solid #1a1a1a" }}>
              <td style={{ padding:"10px", color:"#e8e8e8" }}>{p.name}</td>
              <td style={{ padding:"10px", color:"#555", fontSize:11 }}>{p.
              <td style={{ padding:"10px", color:"#aaa", textAlign:"center"
              <td style={{ padding:"10px", color:"#8080e8" }}>{p.oem_price
              <td style={{ padding:"10px", color:"#20e8a0" }}>{p.aftermarke
              <td style={{ padding:"10px" }}>
                <span style={{ ...S.badge, background: p.critical ? "#2a0a0
                  {p.critical ? "YES" : "NO"}
                </span>
```

```
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </Card>
        <Card title="LABOR ESTIMATE" accent="#a020e8">
          <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:16
            {[["Est. Hours", result.labor_estimate?.hours],["Shop Rate", result.l
              <div key={lbl}>
                <div style={{ ...S.label, marginBottom:4 }}>{lbl}</div>
                <div style={{ color:"#a060e8", fontFamily:"monospace", fontSize:1
              </div>
            ))}
          </div>
        </Card>
        <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12 }}>
          <Card title="WHERE TO BUY" accent="#20e8a0">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.where_to_buy?.map((w
          </Card>
          <Card title="COST-SAVING TIPS" accent="#e8e020">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.cost_saving_tips?.ma
          </Card>
        </div>
      </>}
    </div>
  );
}

// ── Tab 6: Tool Finder ──
function ToolFinder() {
  const [job, setJob] = useState(""); const [year, setYear] = useState(""); const
  const [loading, setLoading] = useState(false); const [result, setResult] = useS
  const catColor = { Basic:"#20a0e8", Specialty:"#e8a020", Power:"#e84040", Safet

  async function handle() {
    if (!job.trim()) return;
    setLoading(true); setResult(null);
    const v = year && make && model ? `${year} ${make} ${model}` : "generic vehic
    try {
      const raw = await callClaude(`Job: ${job}\nVehicle: ${v}`,
        `You are an expert automotive technician. Respond ONLY with JSON: {"job_t
      setResult(extractJSON(raw));
    } catch (e) { setResult({ error:true, msg:e.message }); }
    setLoading(false);
  }
```

```
const skillCfg = { Beginner:["#0a2a0a","#50d050"], Intermediate:["#1a1a0a","#e8

return (
  <div>
    <div style={{ marginBottom:14 }}>
      <label style={S.label}>JOB / REPAIR</label>
      <input style={S.select} placeholder="e.g. brake pad replacement, timing b
        value={job} onChange={e => setJob(e.target.value)} onKeyDown={e => e.ke
    </div>
    <VehicleSelector year={year} setYear={setYear} make={make} setMake={setMake
    <button onClick={handle} disabled={loading || !job.trim()} style={S.btn}>{l
    {loading && <Spinner label="Building tool list..." />}
    {result?.error && <ErrMsg msg={result.msg} />}
    {result && !result.error && <>
      <Card accent="#e8a020">
        <div style={{ display:"flex", justifyContent:"space-between", flexWrap:
          <div>
            <div style={{ color:"#fff", fontSize:18, fontWeight:700, marginBott
            <p style={{ ...S.body, margin:0 }}>{result.overview}</p>
          </div>
          <div style={{ display:"flex", gap:8, alignItems:"flex-start", flexWra
            {(() => { const [bg,fg] = skillCfg[result.skill_level] || ["#1a1a1a
          </div>
        </div>
      </Card>
      <Card title="REQUIRED TOOLS" accent="#20a0e8">
        <div style={{ display:"flex", gap:6, flexWrap:"wrap", marginBottom:14 }
          {Object.entries(catColor).map(([cat,col]) => <span key={cat} style={{
        </div>
        {result.required_tools?.map((t,i) => (
          <div key={i} style={{ display:"flex", gap:14, marginBottom:12, paddin
            <div style={{ width:8, flexShrink:0, background:catColor[t.category
            <div>
              <div style={{ display:"flex", gap:8, alignItems:"center", marginB
                <span style={{ color:"#fff", fontFamily:"monospace", fontSize:1
                {t.spec && <span style={{ color:"#666", fontSize:11, fontFamily
              </div>
              <div style={S.body}>{t.why}</div>
            </div>
          </div>
        ))}
      </Card>
      {result.specialty_tools?.length > 0 && (
        <Card title="SPECIALTY / RENT-OR-BUY TOOLS" accent="#e8a020">
          {result.specialty_tools.map((t,i) => (
            <div key={i} style={{ display:"flex", gap:14, marginBottom:12 }}>
```

```
                <div style={{ width:8, flexShrink:0, background:"#e8a020", border
                <div style={{ flex:1 }}>
                  <div style={{ display:"flex", justifyContent:"space-between", m
                    <span style={{ color:"#fff", fontFamily:"monospace", fontSize
                    <span style={{ ...S.badge, background: t.rental_available ? "
                      {t.rental_available ? "RENT AVAILABLE" : "MUST BUY"}
                    </span>
                  </div>
                  <div style={S.body}>{t.why}</div>
                </div>
              </div>
            ))}
          </Card>
        )}
        <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12 }}>
          <Card title="CONSUMABLES / PARTS" accent="#a020e8">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.consumables?.map((c,
          </Card>
          <Card title="SAFETY GEAR" accent="#20e870">
            <ul style={{ paddingLeft:18, margin:0 }}>{result.safety_gear?.map((s,
          </Card>
        </div>
        <Card title="PRO TIPS" accent="#e8e020">
          {result.pro_tips?.map((t,i) => (
            <div key={i} style={{ display:"flex", gap:10, marginBottom:8 }}>
              <span style={{ color:"#e8e020", flexShrink:0 }}>✦</span><span style
            </div>
          ))}
        </Card>
        <Card title="WHEN TO CALL A PRO" accent="#e84040">
          <p style={{ ...S.body, color:"#ff8080", margin:0 }}>{result.when_to_cal
        </Card>
      </>}
    </div>
  );
}

// ── Tab 7: VIN ──
function VINLookup() {
  const [vin, setVin] = useState(""); const [touched, setTouched] = useState(fals
  const cleanVin = vin.toUpperCase().replace(/[^A-Z0-9]/g, "").slice(0,17);
  const isValid = cleanVin.length === 17;

  const services = [
    { name:"CARFAX", color:"#e84040", badge:"$45 single", desc:"Most comprehensiv
    { name:"AUTOCHECK", color:"#20a0e8", badge:"$25 single", desc:"Strong on auct
    { name:"NHTSA RECALLS", color:"#20e870", badge:"FREE", desc:"Government datab
```

```
  { name:"VIN DECODER", color:"#e8e020", badge:"FREE", desc:"Decode the VIN to
];

return (
  <div>
    <div style={{ marginBottom:14, padding:"12px 16px", background:"#1a1a2a", b
      <div style={{ color:"#8080e8", fontFamily:"monospace", fontSize:11, lette
      <div style={{ ...S.body, fontSize:12 }}>Run a vehicle by VIN to check his
    </div>
    <div style={{ marginBottom:20 }}>
      <label style={S.label}>VIN (17 CHARACTERS)</label>
      <input style={{ ...S.select, textTransform:"uppercase", letterSpacing:2,
        placeholder="1HGBH41JXMN109186" value={vin}
        onChange={e => { setVin(e.target.value); setTouched(false); }} maxLengt
      <div style={{ display:"flex", justifyContent:"space-between", marginTop:6
        <div style={{ fontFamily:"monospace", fontSize:11, color: isValid ? "#2
          {cleanVin.length} / 17 {isValid && "✓"}{touched && !isValid && " — VI
        </div>
        <div style={{ fontFamily:"monospace", fontSize:10, color:"#444" }}>VIN
      </div>
    </div>
    <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:12 }}>
      {services.map(svc => (
        <div key={svc.name} style={{ background:"#0d0d0d", border:`1px solid ${
          <div style={{ position:"absolute", top:0, left:0, right:0, height:2,
          <div style={{ display:"flex", justifyContent:"space-between", alignIt
            <span style={{ color:svc.color, fontFamily:"monospace", fontSize:13
            <span style={{ ...S.badge, background:`${svc.color}22`, color:svc.c
          </div>
          <p style={{ ...S.body, fontSize:12, marginBottom:14 }}>{svc.desc}</p>
          <button onClick={() => { if (!isValid) { setTouched(true); return; }
            style={{ ...S.btn, background:svc.btnColor, color: svc.dark ? "#000
            {svc.btnText}
          </button>
        </div>
      ))}
    </div>
    <Card title="PRO TIPS FOR VEHICLE HISTORY" accent="#e8a020">
      {["Always run BOTH a recall check and a paid history report on any used v
        "Verify the VIN matches in 3 places: dashboard, door jamb sticker, and
        "Carfax misses ~40% of accidents because they only show what's been rep
        "If a seller refuses to share the VIN before viewing the vehicle, walk
        <div key={i} style={{ display:"flex", gap:10, marginBottom:8 }}>
          <span style={{ color:"#e8a020", flexShrink:0 }}>✦</span><span style={
        </div>
      ))}
    </Card>
```

```
    </div>
  );
}

// ── Main App ──
const TABS = [
  { label:"COMPLAINT", icon:"🔧" },
  { label:"DTC CODE", icon:"⚡" },
  { label:"TORQUE", icon:"🔩" },
  { label:"FLUIDS", icon:"🛢️" },
  { label:"PARTS $", icon:"📋" },
  { label:"TOOLS", icon:"🪛" },
  { label:"VIN", icon:"🔍" },
];

export default function App() {
  const [activeTab, setActiveTab] = useState(0);
  const tabComponents = [ComplaintLookup, DTCLookup, TorqueSpecs, FluidCapacity,
  const ActiveComponent = tabComponents[activeTab];

  return (
    <div style={{ minHeight:"100vh", background:"#080808", fontFamily:"'Courier N
      <style>{`
        * { box-sizing: border-box; }
        ::-webkit-scrollbar { width: 6px; } ::-webkit-scrollbar-track { backgroun
        select option { background: #111; }
        button:disabled { opacity: 0.38; cursor: not-allowed; }
        button:not(:disabled):hover { opacity: 0.82; }
        .spinner { width: 16px; height: 16px; border: 2px solid #2a2a2a; border-t
        @keyframes spin { to { transform: rotate(360deg); } }
        input::placeholder, textarea::placeholder { color: #333; }
        input:focus, textarea:focus, select:focus { outline: none; border-color:
        textarea { font-family: 'Courier New', monospace; }
      `}</style>

      {/* Header */}
      <div style={{ borderBottom:"1px solid #1a1a1a", background:"#0a0a0a" }}>
        <div style={{ maxWidth:960, margin:"0 auto", padding:"22px 24px 0" }}>
          <div style={{ fontSize:11, letterSpacing:4, color:"#e8a020", fontWeight
          <div style={{ fontSize:26, fontWeight:700, color:"#fff", letterSpacing:
          <div style={{ fontSize:11, color:"#3a3a3a", marginBottom:18, letterSpac

          {/* Tabs */}
          <div style={{ display:"flex", gap:0, borderTop:"1px solid #1a1a1a", ove
            {TABS.map((tab, i) => (
              <button key={i} onClick={() => setActiveTab(i)} style={{
                background:"none", border:"none", cursor:"pointer", padding:"11px
```

```
                fontFamily:"'Courier New', monospace", fontSize:10, letterSpacing
                color: activeTab === i ? "#e8a020" : "#4a4a4a",
                borderTop: activeTab === i ? "2px solid #e8a020" : "2px solid tra
                marginTop:-1, transition:"color 0.15s", whiteSpace:"nowrap",
              }}>
                {tab.icon} {tab.label}
              </button>
            ))}
          </div>
        </div>
      </div>

      {/* Content */}
      <div style={{ maxWidth:960, margin:"0 auto", padding:"26px 24px 60px" }}>
        <ActiveComponent />
      </div>

      {/* Footer */}
      <div style={{ borderTop:"1px solid #111", padding:"14px 24px", textAlign:"c
        <div style={{ fontSize:10, color:"#252525", letterSpacing:2 }}>
          FOR REFERENCE ONLY · VERIFY WITH FACTORY SERVICE MANUAL · CONSULT A QUA
        </div>
      </div>
    </div>
  );
}
```